

Day 1 Deliverable — Problem Understanding & System Definition

Student: Sreetama Santra | Roll No: 12024052019052 | Date: 2026-08-20
Institute: IEM, Salt Lake, Kolkata | Organization: TIH IIT Guwahati | AI/ML Internship 2026

Part 1: Problem Understanding Note

1.1 What is the Problem?

Electric vehicles depend on a Battery Management System (BMS) to operate safely and efficiently. Two of the most critical parameters the BMS must know in real time are: (1) the State of Charge (SoC) — how much energy is remaining in the battery as a percentage from 0 to 100%, and (2) the Remaining Charging Time — how many minutes are left until the battery reaches full charge. Both of these parameters are difficult to measure directly and must be estimated from sensor readings.

Traditional estimation methods such as Coulomb counting and Extended Kalman Filters have known limitations: they require precise electrochemical models, accumulate errors over time, and do not generalize well across different temperatures, aging states, and usage patterns. Machine learning provides a data-driven alternative that learns the relationship between sensor signals and SoC directly from experimental data, without requiring an explicit model of battery chemistry.

1.2 Why This Problem Matters

Inaccurate SoC estimation leads to real consequences. Overestimating SoC causes unexpected shutdowns when the driver believes more range is available. Underestimating causes premature charging, which accelerates battery degradation. For remaining charging time, poor estimates frustrate users and reduce EV adoption. As EV deployment scales globally, robust and temperature-aware battery prediction systems become essential infrastructure for safe, efficient, and reliable operation.

1.3 Dataset Description — LG HG2 18650 Li-ion Battery Dataset

The dataset used in this project is the LG HG2 18650 Lithium-ion Battery Dataset collected at McMaster University, Hamilton, Ontario, Canada by Dr. Phillip Kollmeyer. It contains experimental measurements from a brand new 3Ah LG HG2 cylindrical Li-ion cell tested under a 75A, 5V Digatron Battery Tester with 0.1% voltage and current accuracy.

The dataset covers six temperature conditions: 40°C, 25°C, 10°C, 0°C, -10°C, and -20°C. At each temperature, a full suite of tests was conducted including drive cycles (UDDS, HWFET, LA92, US06), random mixed drive cycles (Mixed1 to Mixed8), HPPC pulse tests, capacity tests, and charge/discharge cycles. The total dataset contains 211 CSV files.

Column Name	Unit	Physical Meaning
Time Stamp	MM/DD/YYYY HH:MM:SS	Timestamp of each sensor reading at 0.1s resolution for

		drive cycles
Voltage	Volts (V)	Cell terminal voltage measured via sense leads welded to battery terminals
Current	Amperes (A)	Positive during charge, negative during discharge. Key signal for Coulomb counting
Temperature	Celsius (°C)	Battery case temperature at middle of cell, measured with AD592 sensor ($\pm 1^\circ\text{C}$)
Capacity (Ah)	Amp-hours	Cumulative Ah counter, reset after each charge/test. Used to derive SoC labels
WhAccu	Watt-hours	Cumulative energy counter, reset after each test segment

1.4 Input Features for ML Model

The following features will be used as inputs to the machine learning models:

- **Voltage (V)** — rises during charging, drops during discharge. Non-linearly related to SoC, especially at extremes
- **Current (A)** — magnitude and direction indicate charging or discharging rate
- **Temperature (°C)** — significantly affects usable capacity and charging acceptance
- **Capacity / Ah (Ah)** — cumulative charge counter used as a proxy for SoC progression
- **Engineered features:** rolling averages, rate of change of voltage (dV/dt), time elapsed, power ($V \times I$)

1.5 Output Targets

- **State of Charge (SoC)** — derived from the Ah column using Coulomb counting: $\text{SoC} = \text{SoC_initial} + (I \cdot dt) / Q_{\text{nominal}}$. This is a continuous regression target from 0 to 100%.
- **Remaining Charging Time** — computed from timestamps and SoC trajectory during charging sessions. Continuous regression target in minutes.

1.6 Why Are These Targets Hard to Predict?

SoC cannot be sensed directly. Voltage is the most accessible proxy but its relationship with SoC is non-linear and shifts with temperature, aging, and current rate. Coulomb counting (integrating current over time) accumulates sensor noise errors and requires a precisely known starting SoC. The LG HG2 dataset spans six temperatures from -20°C to 40°C , making temperature-robustness a key challenge.

Remaining charging time is non-linear because of the CC-CV transition. During the Constant Current phase, a fixed current enters the cell and voltage rises — SoC increases at a predictable rate. Once voltage hits 4.2V, the Constant Voltage phase begins: current drops gradually as the cell resists

additional charge. The final 20% of charge can take as long as the first 70-80%, making simple linear estimates completely unreliable.

1.7 Why Machine Learning?

ML models learn the complex mapping between sensor readings and SoC from data, without requiring explicit electrochemical models. They can generalize across temperatures, aging states, and drive cycle patterns. Based on the literature reviewed for this project, XGBoost and Random Forest consistently outperform Kalman filter methods on structured battery sensor data. The candidate models planned are: Linear Regression (baseline), Random Forest, XGBoost, and optionally a feedforward ANN.

1.8 Key Terms

- **SoC:** State of Charge — percentage of energy remaining in the battery (0 to 100%)
- **SoH:** State of Health — ratio of current capacity to nominal capacity after cycling
- **OCV:** Open Circuit Voltage — terminal voltage at rest; predictably related to SoC when no current flows
- **DoD:** Depth of Discharge — percentage of capacity used ($\text{DoD} = 100\% - \text{SoC}$)
- **CC Phase:** Constant Current — charger applies fixed current; voltage rises steadily
- **CV Phase:** Constant Voltage — charger holds voltage at 4.2V; current drops as cell fills
- **Coulomb Counting:** Integration of current over time to track cumulative charge; prone to drift
- **HPPC:** Hybrid Pulse Power Characterization — pulse test to characterize internal resistance
- **LG HG2:** LG 18650 HG2 — 3Ah cylindrical Li-ion cell used in this dataset
- **Drive Cycle:** Standard vehicle speed profile (UDDS, HWFET, LA92, US06) used to simulate real EV usage

Part 2: ML Pipeline Diagram

End-to-end workflow from raw sensor data to final evaluated models:

- **Step 1: Raw Data Loading** — Load CSV files from all 6 temperature folders. Parse timestamp, voltage, current, temperature, and Ah columns. Handle variable sample rates (0.1s for drive cycles, slower for charge phases).
- **Step 2: SoC Label Generation** — Derive SoC labels from the Ah column using Coulomb counting: $\text{SoC}(t) = \text{SoC_initial} + \text{cumulative_Ah} / Q_{\text{nominal}} \times 100$. Validate against known charge/discharge capacity.
- **Step 3: Data Inspection and Cleaning** — Check for missing values, corrupted timestamps, negative voltage readings, and sensor dropout periods. Document all anomalies. Apply domain-aware imputation where appropriate.
- **Step 4: Exploratory Data Analysis (EDA)** — Plot voltage, current, temperature vs time for each drive cycle. Visualize SoC trajectories across temperatures. Plot correlation heatmaps. Analyze CC vs CV phase distributions.

- **Step 5: Feature Engineering** — Create derived features: rolling mean of voltage/current (window=10s), dV/dt (rate of voltage change), power ($V \times I$), time since last charge, CC/CV phase flag, temperature category.
- **Step 6: Chronological Train/Val/Test Split** — Split by time within each temperature group. Training: Mixed1-6, Validation: Mixed7, Test: Mixed8 and drive cycles. Never shuffle — leakage prevention is critical for time-series data.
- **Step 7: Model Training** — Train four models: (1) Linear Regression as baseline, (2) Random Forest ($n_estimators=100$), (3) XGBoost with early stopping, (4) ANN if data size justifies.
- **Step 8: Dual-Target Evaluation** — Evaluate each model separately for SoC prediction and remaining charging time. Metrics: MAE, RMSE, R-squared. Compare across temperature conditions (especially -20°C vs 40°C).
- **Step 9: Error and Failure Analysis** — Analyze residuals across SoC ranges (0-20%, 20-80%, 80-100%) and across CC/CV phases. Identify temperature sensitivity of each model. Feature importance analysis via RF and XGBoost.
- **Step 10: Final Model Selection and Report** — Select best model based on validation MAE for SoC. Document results in comparison table. Submit final notebook, report, and trained model weights.

Part 3: Questions List for Mentor

The following questions were identified after studying the problem statement and exploring the LG HG2 dataset:

Dataset Questions

1. The Ah counter in the dataset resets after each test segment. Should SoC be derived independently per segment, or should we track cumulative SoC across a full session?
2. The dataset has variable sample rates — 0.1s for drive cycles and slower for charge phases. Should we upsample to a uniform 0.1s timestep, or handle the two rates separately?
3. Should we train one model across all six temperatures, or train separate models per temperature and compare?
4. The dataset includes both discharge (drive cycles) and charge sessions. Should remaining charging time be predicted only from charging files, or also derived from discharge recovery patterns?
5. Are there specific drive cycles (UDDS, HWFET, LA92, US06, Mixed) that should be prioritized for training versus held for testing?

Modeling Questions

6. Should SoC and remaining charging time be predicted as two separate models, or as a single multi-output regression model?
7. Is there an acceptable SoC error threshold defined for this project — for example, must SoC prediction error be below 3%?

8. Should models be evaluated separately for low SoC (0-20%), mid SoC (20-80%), and high SoC (80-100%) ranges, since behavior differs significantly across these zones?
9. Is model interpretability a priority, or is raw accuracy the primary objective?

Submission Questions

10. For each daily GitHub submission, should I upload only the deliverable document, or also include any supporting notebooks or code?
11. Is the file naming convention WEEK_x_Day_y_SreetamaSantra to be used for all files including notebooks and PDFs?